

Enhancing Retrieval Quality in RAG-Based Defensive Security Systems through Query Reformulation

Subhodeep Sarkar – 253000118 subhodeep25300@iiitnr.edu.in

Nitish Kumar Bind Sarkar – 253000112 nitish25300@iiitnr.edu.in

Anurag Kumar Sahu – 2530001104 anurag25300@iiitnr.edu.in

Dr. S.P.M International Institute of Information Technology, Naya Raipur, CG

Abstract—

This paper investigates whether large-language-model-based query reformulation can improve retrieval quality in defensive security retrieval-augmented generation systems. The project targets analyst-facing cybersecurity scenarios where users often ask vague questions such as 'something weird in my network traffic' while the underlying corpus contains structured terminology from MITRE ATT&CK, NVD CVE records, and CISA advisories. We build a baseline RAG pipeline and four reformulation pipelines: query expansion, hypothetical document embedding (HyDE), step-back prompting, and sub-query decomposition. The system is implemented locally with Ollama using Llama 3.2 for generation and nomic-embed-text for embeddings, with ChromaDB as the persistent vector store. Evaluation over 15 security queries -- 8 vague and 7 specific -- shows that no single pipeline dominates all metrics. HyDE achieves the best overall Recall@5 (0.259) and MRR (0.600), while Decomposition produces the most grounded answers with the highest Faithfulness (0.529). Critically, the best pipeline depends on query type: Expansion performs best on vague queries (Recall@5 = 0.150) while HyDE dominates specific queries (Recall@5 = 0.419, MRR = 0.857). Step-Back prompting is consistently the weakest strategy. The findings support a selective routing policy for real-world security systems.

Keywords—retrieval-augmented generation, cybersecurity, query reformulation, defensive security, MITRE ATT&CK, vector search, Ollama

I. INTRODUCTION

Modern defensive security operations increasingly depend on fast access to large bodies of structured and semi-structured knowledge, including threat-intelligence reports, MITRE ATT&CK techniques, vulnerability records, and remediation advisories. Retrieval-Augmented Generation (RAG) offers a practical way to combine semantic retrieval with large language model reasoning so that an analyst can ask a natural-language question and receive an answer grounded in supporting evidence.

The central problem addressed in this project is query-document mismatch. Security analysts and students often phrase questions in vague, operational, or symptom-based language, whereas the indexed corpus is written in precise domain terminology. For example, a user may ask about “weird network traffic,” while the relevant knowledge-base entries use terms such as command-and-control, network service discovery, lateral movement, or exfiltration. When this mismatch occurs, retrieval quality drops and the final answer becomes weak or unsupported.

This project therefore studies the following research question: *Does query reformulation using LLM-based agents improve retrieval relevance and final answer quality in defensive security RAG systems?*

To answer this question, we designed and evaluated a full local cybersecurity RAG system. The system ingests defensive security knowledge, embeds it into a vector database, supports baseline and reformulated retrieval paths, and measures performance using retrieval and grounding metrics across a mixed benchmark of vague and specific security questions.

II. PROJECT GOAL AND SYSTEM OVERVIEW

A. What We Are Trying to Build

The project goal is to build an agentic AI assistant for defensive security. A user asks a cybersecurity question, the system retrieves relevant knowledge from a security corpus, and an LLM generates an answer from those retrieved documents. The experimental contribution is the addition of a query reformulation layer that tries to improve the search step before answer generation.

B. Knowledge Base and Document Store

The knowledge base combines three public cybersecurity sources that are realistic and manageable for a student research project. First, MITRE ATT&CK provides structured adversary techniques, tactics, platforms, and descriptions. Second, the National Vulnerability Database provides CVE records and vulnerability descriptions. Third, the CISA Known Exploited Vulnerabilities catalog provides remediation-oriented entries for vulnerabilities observed in real-world exploitation. These sources were chunked and embedded into a persistent Chroma vector store.

C. End-to-End Workflow

The full workflow has four stages. Stage 1 ingests and chunks the cybersecurity documents. Stage 2 embeds those chunks and stores them for dense retrieval. Stage 3 runs either the baseline or a reformulation strategy to retrieve top-k evidence. Stage 4 prompts the LLM to answer using the retrieved context, after which benchmark metrics are computed.

III. PIPELINE DESIGN

Five pipelines were evaluated in this project. The baseline serves as the control condition, and four enhanced pipelines apply different query-rewriting strategies before retrieval. Table I summarizes their behavior, and the following subsections explain each one clearly.

TABLE I

Pipelines Evaluated in the RAGDMS System

Pipeline	What It Does
Baseline	Uses the raw user query exactly as typed, retrieves the top security chunks, and generates an answer from those retrieved documents.
Expansion	Rewrites the raw query by adding cybersecurity language such as attack behaviors, MITRE-style terms, and detection-oriented wording.
HyDE	Generates a hypothetical answer passage first, then embeds that passage as the retrieval query so the search resembles the language of the corpus.
Step-Back	Generalizes the question to a broader security concept before retrieval so the model can pull more foundational context.
Decomposition	Breaks a complex query into smaller sub-questions, retrieves evidence for each one separately, then merges the evidence for answer generation.

A. Baseline Pipeline

The baseline pipeline is the simplest control system. A user query is embedded directly with nomic-embed-text, the top retrieved documents are loaded from ChromaDB, and Llama 3.2 answers using that evidence. This pipeline is important because it shows how well a normal RAG system performs without any extra intelligence in the query stage.

B. Query Expansion

Query expansion attempts to enrich an underspecified query with security terminology. The purpose is to reduce the lexical gap between human phrasing and security documentation. For example, a vague report about unusual logins or network anomalies can be reframed using concepts such as brute force, credential stuffing, lateral movement, beaconing, or exfiltration. The intended benefit is better recall when the user does not know the correct technical phrase.

C. Hypothetical Document Embedding (HyDE)

HyDE does not merely rewrite the question. Instead, it asks the LLM to generate a short hypothetical answer as though it were already a snippet from a relevant security article. That generated paragraph is then embedded and used for retrieval. The intuition is that a hypothetical article-like passage is often closer to the structure and vocabulary of real indexed documents than a short user query is.

D. Step-Back Prompting

Step-back prompting transforms a narrow query into a broader conceptual question. The system tries to retrieve more general supporting knowledge that may still help answer the original problem. This can be useful when the

original question is too narrow, too sparse, or expressed in a way that does not match exact corpus entries.

E. Sub-Query Decomposition

Sub-query decomposition is designed for multi-part questions. Instead of sending one broad request to the retriever, the system breaks the query into smaller answerable units. Each sub-query retrieves its own evidence, and the results are merged. In principle, this should improve coverage for compound questions such as detection-plus-response or root-cause-plus-mitigation scenarios.

IV. EXPERIMENTAL SETUP

A. Implementation Stack

Component	Configuration Used
LLM	Llama 3.2 via Ollama
Embedding model	nomic-embed-text via Ollama
Vector store	ChromaDB persistent store
Framework	LangChain-based project structure with custom ingestion and evaluation modules
Corpus sources	MITRE ATT&CK, NVD CVE data, and CISA KEV advisories
Benchmark queries	15 total: 8 vague queries and 7 specific queries
Retrieval depth	Top-5 documents per query

B. Benchmark Dataset

The benchmark dataset contains 15 cybersecurity questions with manually written ground-truth answers and keyword sets. Eight questions are vague, symptom-style analyst queries such as “something weird in my network traffic” or “admin account doing things at 3am.” Seven are specific technical questions such as “How does Pass-the-Hash work?” and “How does DMARC protect against email spoofing?” This split allows the paper to study not only whether reformulation helps, but also when it helps.

C. Evaluation Metrics

Four metrics were used. Recall@5 measures whether the retriever found relevant material within the top five documents. MRR measures how early the first relevant result appears in the ranking. Answer overlap is a lexical proxy for answer accuracy relative to the prepared ground-truth answer. Faithfulness measures whether the generated answer appears grounded in the retrieved context rather than unsupported model memory.

V. RESULTS

Table II reports the mean scores across all 15 benchmark queries. HyDE achieved the highest Recall@5 (0.259) and MRR (0.600), making it the best overall retrieval pipeline. Decomposition achieved the highest Faithfulness (0.529), meaning its answers stay closest to the retrieved documents. Step-Back is consistently the weakest across all metrics. Importantly, no single pipeline wins on every metric — the best choice depends on the query type and what the system optimizes for.

TABLE II
Mean Benchmark Scores Across All 15 Queries

Pipeline	Recall@5	MRR	Ans. Overlap	Faithfulness
Baseline	0.177	0.400	0.086	0.302
Expansion	0.167	0.417	0.066	0.338
HyDE	0.259	0.600	0.085	0.375
Step-Back	0.114	0.300	0.057	0.206
Decomposition	0.172	0.372	0.075	0.529

TABLE III
Recall@5 by Query Category

Pipeline	Vague Recall@5	Specific Recall@5	Verdict
Baseline	0.056	0.314	Fails on vague; solid on specific
Expansion	0.150	0.186	Best for vague queries
HyDE	0.119	0.419	Best for specific queries
Step-Back	0.088	0.145	Consistently worst
Decomposition	0.119	0.233	High faithfulness (0.529) but mid recall

A. What the Results Actually Show

The results reveal three distinct findings. First, HyDE is the best overall retrieval pipeline (Recall@5 = 0.259, MRR = 0.600) because its hypothetical passage naturally matches the language of security documents. Second, Decomposition produces the most grounded answers (Faithfulness = 0.529) by retrieving evidence for each sub-question separately before synthesis. Third, Step-Back is consistently the weakest strategy across all metrics, suggesting that over-generalizing security queries removes the precision needed for effective retrieval.

Table III reveals the most actionable finding: the best pipeline depends entirely on query type. The baseline completely fails on vague queries (Recall@5 = 0.056 — nearly zero), while Expansion rescues vague queries with a Recall@5 of 0.150. For specific technical queries, HyDE dominates with Recall@5 = 0.419 and MRR = 0.857, meaning it almost always ranks the correct document first. This confirms the practical recommendation: a smart security system should classify the query type first and route accordingly.

B. Qualitative Examples from the Demo

The screenshots captured during the local demonstration illustrate why the quantitative results matter. In some vague cases, reformulation produces a clearer retrieval path and a more security-aware explanation. In other cases, especially when the model invents overly specific or incorrect details, reformulation can push retrieval away from the correct evidence.

```

=====
DEMO CASE: Vague Query - "users getting locked out randomly?"
=====
BASELINE - Raw Query: "users getting locked out randomly?"
Answer:
Based on the provided context, it appears that the issue of users getting locked out randomly could be related to brute force attacks on WNC libraries
(s/Implementations). The citations mention specific vulnerabilities such as "hijacking WNC" and "weak root WNC login without authentication", which suggest
t that unauthorized access attempts can lead to user lockouts.

It is possible that the issue of users getting locked out randomly is due to an attacker attempting to brute force their way into a WNC session, caus
ing the account to be locked out after a certain number of failed login attempts. This would require further investigation and analysis to determine the
root cause and implement appropriate mitigations to prevent such attacks.

However, without more specific information about the environment, such as the operating system, version of the WNC library, or any other relevant det
ails, it is difficult to provide a more precise answer.
Recall@5=0.25 MRR=1.00 Faithfulness=0.20

=====
ENHANCED [EXPANSION]
Reformulated: "Are there any instances of brute-force attacks, session hijacking, or credential stuffing occurring on our system, poten
Answer:
Based on the provided context, users getting locked out randomly could be a sign of brute forcing credentials. Adversaries may attempt to brute force
access to valid accounts within a victim environment, leveraging knowledge gathered from other post-compromise behaviors such as OS Credential Dumping,
Account Discovery, or Password Policy Discovery.

This behavior is consistent with the tactics, techniques, and procedures (TTPs) described in the context, where adversaries use various techniques to
gather information about valid accounts and then attempt to brute force access to them. Therefore, if users are getting locked out randomly, it could be
a indication of an adversary attempting to brute force credentials within the environment.
Recall@5=0.25 MRR=1.00 Faithfulness=0.50

```

Fig. 1. Vague query: 'users getting locked out'. Baseline (Recall=0.25) vs. HyDE (Recall=0.50). HyDE generates a brute-force-oriented passage that matches ATT&CK T1110.

```

=====
DEMO CASE: Specific Query - "What MITRE ATTACK techniques are used in ransomware attacks?"
=====
BASELINE - Raw Query: "What MITRE ATTACK techniques are used in ransomware attacks?"
Answer:
The MITRE ATTACK techniques used in ransomware attacks include:
1. (Data Destruction)(https://attack.mitre.org/techniques/T1485)
2. (Defacement)(https://attack.mitre.org/techniques/T1491)
3. (Data Encrypted for Impact)(https://attack.mitre.org/techniques/T1486)
4. (Exfiltration)(https://attack.mitre.org/techniques/T1059)

These techniques are often used by adversaries to impede incident response and recovery before completing the ransomware attack, such as demanding pa
yment from a victim after encrypting their data and threatening to leak sensitive information unless payment is made.
Recall@5=0.75 MRR=1.00 Faithfulness=1.00

=====
ENHANCED [EXPANSION]
Reformulated: "What specific MITRE ATTACK techniques are employed during a ransomware attack, such as exfiltration of sensitive data (*)"
Answer:
The MITRE ATTACK techniques used in ransomware attacks include:
1. (Data Encrypted for Impact)(https://attack.mitre.org/techniques/T1486)
2. (Defacement)(https://attack.mitre.org/techniques/T1491)
3. (Data Destruction)(https://attack.mitre.org/techniques/T1485)
4. (Defacement)(https://attack.mitre.org/techniques/T1491)

These techniques are used in various combinations to carry out ransomware attacks, which may involve demanding payment from a victim after encrypting
their data, threatening to leak sensitive data unless payment is made, and rendering data permanently inaccessible if the key is not saved or transmitted.

```

Fig. 2. Specific query: MITRE ATT&CK ransomware techniques — Baseline achieves Recall=0.75, MRR=1.00. HyDE hallucinated wrong technique IDs and scored Recall=0.00.

SUMMARY - Mean Scores Across All Demo Queries				
Pipeline	Recall@5	MRR	Faithfulness	
baseline	0.39	0.55	0.46	← control
enhanced_expansion	0.39	0.38	0.47	
enhanced_hyde	0.12	0.25	0.28	
enhanced_stepback	0.25	0.33	0.55	

KEY FINDING:
Reformulation helps most on vague queries.
HyDE and Query Expansion tend to outperform baseline on recall.
Specific queries sometimes perform equally well without reformulation.

subh0deepsarkar@MBA4-5GFF RAGDMS %

Fig. 3. Final benchmark summary table from terminal showing mean scores across all 15 queries and 5 pipelines.

VI. DISCUSSION

A. Why HyDE Worked Best Overall

HyDE worked well because the LLM generates a hypothetical answer written in the same style as security knowledge base documents — structured, technical, and terminology-rich. This makes the embedding of the hypothetical passage semantically close to real indexed entries. HyDE is especially effective for specific queries (Recall@5 = 0.419, MRR = 0.857) because precise security questions have well-defined answers that the LLM can approximate faithfully. The risk is hallucination: if the LLM invents incorrect technique IDs, the retrieval fails entirely.

B. Why Expansion Wins on Vague Queries

Query Expansion is the best strategy for vague, symptom-based queries (Recall@5 = 0.150 vs. baseline 0.056). When a user asks about 'weird network traffic,' expansion adds terms like lateral movement, C2 beaconing, and port scanning — bridging the gap between informal language and security corpus terminology. However, expansion can suffer from prompt drift when the local LLM generates SQL-like or overly structured reformulations,

which degraded performance in earlier runs before prompt engineering corrections.

C. Why Decomposition Has the Best Faithfulness

Decomposition achieves the highest faithfulness (0.529) because it retrieves separate evidence for each sub-question before synthesis. The generated answer is therefore composed from multiple grounded retrieval results rather than relying on model memory. This makes decomposition ideal when answer verifiability is more important than retrieval speed, a relevant trade-off in security audit and compliance scenarios.

D. Why Step-Back Consistently Fails

Step-Back prompting was the weakest strategy across both vague and specific queries. Generalizing security questions loses the precise terminology needed for effective retrieval. For example, stepping back from 'How does Pass-the-Hash work?' to 'How do attackers use stolen credentials?' retrieves broad credential-related documents rather than the specific NTLM/NTLM lateral movement entries that answer the original question.

C. Limitations

This is a compact research-based benchmark, so several limitations remain. The query set contains 15 examples rather than a large industrial testbed. So, the answer overlap is only a proxy for full factual correctness. Ground-truth relevance is based on prepared keyword sets rather than expert annotation. Finally, the system uses a local lightweight model through Ollama, so larger models may change the quality of reformulation.

VII. CONCLUSION

This paper presented a complete defensive security RAG prototype and evaluated whether LLM-based query reformulation improves retrieval quality across five pipelines and 15 benchmark queries. The answer is clear but nuanced: no single pipeline is best for everything. HyDE achieves the highest overall Recall@5 (0.259) and MRR (0.600), Expansion is best for vague queries (Recall@5 = 0.150), HyDE dominates specific queries (Recall@5 = 0.419, MRR = 0.857), Decomposition produces the most grounded answers (Faithfulness = 0.529), and Step-Back is consistently the weakest strategy. To the best of our knowledge, this is among the first studies to evaluate these reformulation strategies specifically on defensive security knowledge bases with vague versus specific analyst queries. The key practical recommendation is a selective routing policy: classify the query type first, then apply the appropriate pipeline rather than applying reformulation blindly.

Future work can extend this project with a larger labeled benchmark, log-centric datasets, stronger local models, and an adaptive controller that chooses among baseline, HyDE, expansion, or decomposition based on the incoming question.

ACKNOWLEDGMENT

This paper is based on the implementation, benchmarking, and local demonstration carried out in the RAGDMS project workspace. The work used public security knowledge sources and local Ollama-based inference to preserve an offline experimental workflow.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuhn, V. Stoyanov, W. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020.
- [2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.
- [3] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise zero-shot dense retrieval without relevance labels," in Proc. ACL, 2023.
- [4] H. Zheng, Z. Feng, Z. Xie, and J. Zhou, "Take a step back: Evoking reasoning via abstraction in large language models," arXiv:2310.06117, 2023.
- [5] MITRE, "ATT&CK knowledge base," 2026. [Online]. Available: <https://attack.mitre.org>
- [6] NIST, "National vulnerability database," 2026. [Online]. Available: <https://nvd.nist.gov>
- [7] CISA, "Known exploited vulnerabilities catalog," 2026. [Online]. Available: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>